

TorchTEM Layer Equations

tspi

1 Purpose

This document records the mathematical form of implemented layers in `torchtem`.

2 Notation

We use the following notation:

$$\psi(\mathbf{r}) \in \mathbb{C} \quad \text{complex wave in real space,} \quad (1)$$

$$\Psi(\mathbf{k}) \in \mathbb{C} \quad \text{complex wave in reciprocal space,} \quad (2)$$

$$\mathbf{r} = (x, y), \quad (3)$$

$$\mathbf{k} = (k_x, k_y), \quad (4)$$

$$\lambda = \lambda(E) \quad \text{relativistic electron wavelength,} \quad (5)$$

$$\sigma = \sigma(E) \quad \text{electron interaction parameter,} \quad (6)$$

$$V_j(\mathbf{r}) \quad \text{projected electrostatic potential of slice } j. \quad (7)$$

The forward and inverse Fourier transforms in the current implementation use orthonormal FFT normalization:

$$\Psi(\mathbf{k}) = \mathcal{F}\{\psi(\mathbf{r})\}, \quad (8)$$

$$\psi(\mathbf{r}) = \mathcal{F}^{-1}\{\Psi(\mathbf{k})\}. \quad (9)$$

3 Wave Containers and Builders

3.1 Waves

The Torch-native `Waves` container stores an array

$$\psi \in \mathbb{C}^{n_1 \times \dots \times n_m \times N_y \times N_x}, \quad (10)$$

where the final two indices are spatial dimensions and the leading indices are optional ensemble dimensions.

The real-space intensity and phase observables are

$$I(\mathbf{r}) = |\psi(\mathbf{r})|^2, \quad (11)$$

$$\phi(\mathbf{r}) = \arg \psi(\mathbf{r}). \quad (12)$$

Reciprocal-space conversion is defined by

$$\Psi(\mathbf{k}) = \mathcal{F}\{\psi(\mathbf{r})\}, \quad (13)$$

and real-space reconstruction from reciprocal representation is

$$\psi(\mathbf{r}) = \mathcal{F}^{-1}\{\Psi(\mathbf{k})\}. \quad (14)$$

Two normalization modes are currently implemented.

Value normalization.

$$\psi_{\text{norm}}(\mathbf{r}) = \frac{\psi(\mathbf{r})}{\max_{\mathbf{r}} |\psi(\mathbf{r})| + \varepsilon}. \quad (15)$$

Reciprocal-space normalization.

$$\psi_{\text{norm}}(\mathbf{r}) = \frac{\psi(\mathbf{r})}{(\sum_{\mathbf{k}} |\Psi(\mathbf{k})|^2)^{1/2} + \varepsilon}. \quad (16)$$

3.2 PlaneWaveBuilder

The plane-wave builder emits a constant incident field

$$\psi_0(\mathbf{r}) = A, \quad (17)$$

where $A \in \mathbb{C}$ is a user-selected complex amplitude.

3.3 ProbeWavesBuilder

The probe builder wraps the probe-forming optics and returns

$$\psi_{\text{probe}}(\mathbf{r}) = \mathcal{F}^{-1}\{H(\mathbf{k}) A(\mathbf{k})\}, \quad (18)$$

where $A(\mathbf{k})$ is an optional reciprocal-space amplitude mask and $H(\mathbf{k})$ is the transfer function defined below.

4 Optics

4.1 CTF

The contrast transfer function is modeled as

$$H(\mathbf{k}) = P(\mathbf{k}) \exp[-i\chi(\mathbf{k})] E_t(\mathbf{k}) E_s(\mathbf{k}), \quad (19)$$

where P is the aperture, χ is the aberration phase, E_t is the temporal envelope, and E_s is the spatial envelope. In the implementation, E_t and E_s are included only when the corresponding spread parameters are non-zero.

Relation to physical lens and aperture fields. The implemented CTF is a reduced effective optics model rather than a direct electromagnetic field solve inside the microscope column. The intended bridge to hardware is

$$(\mathbf{E}(\mathbf{r}), \mathbf{B}(\mathbf{r}), \text{aperture geometry}) \implies \text{optical path / eikonal phase} \implies \chi(\mathbf{k}), P(\mathbf{k}). \quad (20)$$

At the Lorentz-force level, an electron ray or wave packet is driven by

$$\frac{d\mathbf{p}}{dt} = -e(\mathbf{E} + \mathbf{v} \times \mathbf{B}), \quad (21)$$

and, after the usual paraxial and reference-trajectory reduction, the accumulated optical-path phase may be written schematically as

$$\Phi(\boldsymbol{\rho}) \approx \Phi_0 + \sum_j a_j f_j(\boldsymbol{\rho}), \quad (22)$$

where $\boldsymbol{\rho}$ is a pupil-plane coordinate and the coefficients a_j are low-order summaries of the underlying lens fields. In reciprocal-space TEM notation, this reduced phase is then represented by the aberration function

$$\chi(\mathbf{k}) = \Phi(\boldsymbol{\rho}(\mathbf{k})) - \Phi_0, \quad (23)$$

which is finally expanded in the aberration basis used by the simulator. A real aperture enters the same reduced model as the pupil support or amplitude mask

$$P(\mathbf{k}) \in [0, 1], \quad (24)$$

with hard, soft, annular, segmented, or otherwise parameterized support depending on the experiment branch. So, in the current TorchTEM implementation, the aberration coefficients and aperture masks should be interpreted as compressed observables of the physical lens and aperture fields, not as a first-principles field map of the full column.

Let $\alpha = 10^3 \lambda |\mathbf{k}|$ be the scattering angle in mrad and φ the polar angle of $\mathbf{k}$. The aberration phase polynomial is represented as

$$\chi(\alpha, \varphi) = \frac{2\pi}{\lambda} \sum_{n,m} \frac{\alpha^{n+1}}{n+1} C_{nm} \cos[m(\varphi - \phi_{nm})], \quad (25)$$

with explicit implemented terms up to fifth order.

The hard or soft aperture is

$$P(\alpha) = \begin{cases} \mathbf{1}\{\alpha \leq \alpha_c\}, & \text{hard cutoff,} \\ \sigma\left(\frac{\alpha_c - \alpha}{s}\right), & \text{soft edge,} \end{cases} \quad (26)$$

where α_c is the semiangle cutoff, s is the soft-edge width, and σ is the logistic function.

The temporal envelope is

$$E_t(\alpha) = \exp\left[-\left(\frac{\pi}{2\lambda} \Delta f \alpha^2\right)^2\right], \quad (27)$$

where Δf is the focal spread.

5 Multislice Propagation

5.1 Transmission

For slice j , the transmission function is

$$T_j(\mathbf{r}) = \exp[i\sigma V_j(\mathbf{r})]. \quad (28)$$

5.2 Fresnel propagation

Between slices, the current implementation uses an exact Fourier-space propagator

$$\Psi_{j+1}(\mathbf{k}) = G(\mathbf{k}; \Delta z) \Psi'_j(\mathbf{k}), \quad (29)$$

where Δz is the slice thickness and

$$\Psi'_j(\mathbf{k}) = \mathcal{F} \{T_j(\mathbf{r})\psi_j(\mathbf{r})\}. \quad (30)$$

The propagated wave in real space is

$$\psi_{j+1}(\mathbf{r}) = \mathcal{F}^{-1} \{G(\mathbf{k}; \Delta z) \mathcal{F} [T_j(\mathbf{r})\psi_j(\mathbf{r})]\}. \quad (31)$$

The implemented exact propagator distinguishes propagating and evanescent components via $x = \lambda^2|\mathbf{k}|^2$:

$$G(\mathbf{k}; \Delta z) = \begin{cases} \exp \left[\frac{2\pi\Delta z}{\lambda} \left(-\frac{x}{\sqrt{1-x}+1} \right) \right], & x \leq 1, \\ \exp \left[\frac{2\pi\Delta z}{\lambda} (i\sqrt{x-1}-1) \right], & x > 1. \end{cases} \quad (32)$$

5.3 Multislice recursion

Starting from incident wave $\psi_0(\mathbf{r})$, the full multislice recursion is

$$\psi_{j+\frac{1}{2}}(\mathbf{r}) = T_j(\mathbf{r})\psi_j(\mathbf{r}), \quad (33)$$

$$\psi_{j+1}(\mathbf{r}) = \mathcal{F}^{-1} \left\{ G(\mathbf{k}; \Delta z) \mathcal{F} [\psi_{j+\frac{1}{2}}(\mathbf{r})] \right\}. \quad (34)$$

6 Wave–Optics Interaction

Applying a CTF to a `Waves` object is defined as:

$$\Psi_{\text{out}}(\mathbf{k}) = H(\mathbf{k}) \Psi_{\text{in}}(\mathbf{k}), \quad (35)$$

where Ψ_{in} is either the stored reciprocal-space wave, or the Fourier transform of a stored real-space wave. If the input container is in real space, the output is returned to real space after multiplication:

$$\psi_{\text{out}}(\mathbf{r}) = \mathcal{F}^{-1} \{H(\mathbf{k})\mathcal{F}\{\psi_{\text{in}}(\mathbf{r})\}\}. \quad (36)$$

7 Specimen and Potential Layers

7.1 GaussianAtomProjection

The current smooth surrogate projected-potential layer stores atom positions $\mathbf{r}_n = (y_n, x_n)$, amplitudes a_n , and widths σ_n and produces

$$V(\mathbf{r}) = \sum_{n=1}^{N_{\text{atom}}} a_n \exp\left(-\frac{\|\mathbf{r} - \mathbf{r}_n\|^2}{2\sigma_n^2}\right). \quad (37)$$

This layer is used as a trainable, differentiable surrogate while more faithful parametrized atomic potentials are also available elsewhere in the implementation.

7.2 Projected potential slices

When a single projected potential $V(\mathbf{r})$ is reused across a multislice stack of N_s equal slices, the current TEMModel path distributes it as

$$V_j(\mathbf{r}) = \frac{1}{N_s} V(\mathbf{r}), \quad j = 1, \dots, N_s. \quad (38)$$

If the potential module already returns a rank-3 tensor of slice-resolved potentials, that tensor is interpreted directly as

$$\{V_j(\mathbf{r})\}_{j=1}^{N_s}. \quad (39)$$

7.3 IAM slice builder

The current IAM potential builder stores atoms

$$\mathbf{R}_n = (y_n, x_n, z_n), \quad (40)$$

chemical symbols s_n , and optional amplitudes a_n , and returns a slice stack

$$\{V_j(\mathbf{r})\}_{j=1}^{N_s}, \quad [z_j^-, z_j^+] \subset [0, L_z]. \quad (41)$$

For infinite projected-potential mode, each atom is assigned to exactly one slice by

$$m_j(n) = \mathbf{1}\{z_n \in [z_j^-, z_j^+]\}, \quad (42)$$

and the slice potential is assembled as

$$V_j(\mathbf{r}) = \sum_{n=1}^{N_{\text{atom}}} m_j(n) a_n \Phi_{s_n}^\infty(\mathbf{r} - \mathbf{R}_{n,xy}), \quad (43)$$

where $\Phi_{s_n}^\infty$ denotes the chosen infinite projected atomic potential, either from the direct parametrized projected-potential formulas or from the projected-scattering-factor integrator path.

For finite projected mode, each slice accumulates the finite z -window contribution of every atom,

$$V_j(\mathbf{r}) = \sum_{n=1}^{N_{\text{atom}}} a_n \Phi_{s_n}^{[z_j^-, z_j^+]}(\mathbf{r} - \mathbf{R}_{n,xy}, z_n), \quad (44)$$

where $\Phi_{s_n}^{[z_j^-, z_j^+]}$ is evaluated either by the explicit finite Peng formula or by the Gaussian-plus-correction projection-integral route used for the current Lobato/Kirkland finite builder support.

8 Scan Layers

8.1 Fourier-shift scan positioning

Given a reference wave $\psi(\mathbf{r})$ and scan shift $\Delta\mathbf{r}_m = (\Delta y_m, \Delta x_m)$, the implementation applies the Fourier shift theorem:

$$\psi_m(\mathbf{r}) = \mathcal{F}^{-1} \{ \exp[-2\pi i (\Delta y_m k_y + \Delta x_m k_x)] \mathcal{F}\{\psi(\mathbf{r})\} \}. \quad (45)$$

8.2 GridScan

For scan start point $\mathbf{r}_{\min} = (y_{\min}, x_{\min})$, end point $\mathbf{r}_{\max} = (y_{\max}, x_{\max})$, and raster shape $(N_y^{\text{scan}}, N_x^{\text{scan}})$, the scan positions are

$$y_i = y_{\min} + \frac{i}{N_y^{\text{scan}} - 1} (y_{\max} - y_{\min}), \quad (46)$$

$$x_j = x_{\min} + \frac{j}{N_x^{\text{scan}} - 1} (x_{\max} - x_{\min}), \quad (47)$$

with lexicographically flattened position list

$$\mathcal{S} = \{(y_i, x_j)\}_{i,j}. \quad (48)$$

8.3 CustomScan

The custom scan layer is a direct position map:

$$\mathcal{S} = \{(y_m, x_m)\}_{m=1}^M, \quad (49)$$

where the full list is supplied by the caller.

9 Detection Layers

9.1 DiffractionIntensity

The diffraction intensity operator computes

$$I(\mathbf{k}) = |\mathcal{F}\{\psi(\mathbf{r})\}|^2. \quad (50)$$

9.2 AnnularDetector

Let the angle components in mrad be

$$\alpha_y(\mathbf{k}) = 10^3 \lambda k_y, \quad (51)$$

$$\alpha_x(\mathbf{k}) = 10^3 \lambda k_x, \quad (52)$$

and let $\Delta\alpha = (\Delta\alpha_y, \Delta\alpha_x)$ be an optional detector-center offset. The shifted radial coordinate is

$$\alpha_{\Delta}(\mathbf{k}) = \sqrt{(\alpha_y(\mathbf{k}) - \Delta\alpha_y)^2 + (\alpha_x(\mathbf{k}) - \Delta\alpha_x)^2}. \quad (53)$$

For inner and outer detector radii α_{in} and α_{out} , the annular mask is

$$M_{\text{ann}}(\mathbf{k}) = \mathbf{1}\{\alpha_{\text{in}} \leq \alpha_{\Delta}(\mathbf{k}) < \alpha_{\text{out}}\}, \quad (54)$$

and the detector output is

$$D_{\text{ann}} = \sum_{\mathbf{k}} I(\mathbf{k}) M_{\text{ann}}(\mathbf{k}). \quad (55)$$

The annular detector-region helper returns the binary mask $M_{\text{ann}}(\mathbf{k})$ itself for geometry inspection.

9.3 PixelatedDetector

The pixelated detector may either record diffraction intensity

$$D_{\text{pix}}(\mathbf{k}) = I(\mathbf{k}). \quad (56)$$

or real-space wave intensity

$$D_{\text{pix,real}}(\mathbf{r}) = |\psi(\mathbf{r})|^2. \quad (57)$$

If a target output grid $\mathbf{N}'$ is requested, the current Torch-native implementation resamples the intensity image to that shape using the shared FFT-based interpolation helper. If a maximum detected scattering angle α_{max} is configured in reciprocal mode, the detector first applies the mask

$$M_{\text{max}}(\mathbf{k}) = \mathbf{1}\{\alpha(\mathbf{k}) \leq \alpha_{\text{max}}\}, \quad (58)$$

and returns

$$D_{\text{pix,max}}(\mathbf{k}) = I(\mathbf{k}) M_{\text{max}}(\mathbf{k}). \quad (59)$$

The corresponding detector-region helper returns $M_{\text{max}}(\mathbf{k})$ directly; in real-space mode it returns the all-ones mask because the full image plane is accepted.

9.4 SegmentedDetector

Let the valid angular interval be

$$\alpha_{\text{in}} \leq \alpha_{\Delta}(\mathbf{k}) < \alpha_{\text{out}}, \quad (60)$$

and let the shifted polar angle be

$$\varphi_{\Delta}(\mathbf{k}) = \text{mod}(\text{atan2}(\alpha_x(\mathbf{k}) - \Delta\alpha_x, \alpha_y(\mathbf{k}) - \Delta\alpha_y) - \varphi_0, 2\pi), \quad (61)$$

where φ_0 is the detector rotation. The radial and azimuthal bin indices are

$$b_r(\mathbf{k}) = \left\lfloor N_r \frac{\alpha_{\Delta}(\mathbf{k}) - \alpha_{\text{in}}}{\alpha_{\text{out}} - \alpha_{\text{in}}} \right\rfloor, \quad (62)$$

$$b_{\varphi}(\mathbf{k}) = \left\lfloor N_{\varphi} \frac{\varphi_{\Delta}(\mathbf{k})}{2\pi} \right\rfloor. \quad (63)$$

The segment index is

$$b(\mathbf{k}) = b_r(\mathbf{k}) N_{\varphi} + b_{\varphi}(\mathbf{k}), \quad (64)$$

and the detector output accumulates intensity over each segment:

$$D_{r,\varphi} = \sum_{\mathbf{k} \in \Omega_{r,\varphi}} I(\mathbf{k}). \quad (65)$$

The segmented detector-region helper returns the family of masks $\{\mathbf{1}_{\Omega_{r,\varphi}}(\mathbf{k})\}_{r,\varphi}$, represented with NaN outside each region so per-region plots remain visually separated.

10 Coherence Averaging

The current source-offset ensemble path evaluates multiple coherent members

$$\{\psi^{(m)}(\mathbf{r})\}_{m=1}^M \quad (66)$$

and averages their measured intensities:

$$\bar{D} = \frac{1}{M} \sum_{m=1}^M D[\psi^{(m)}]. \quad (67)$$

This corresponds to an incoherent mixture approximation for the selected source ensemble.

11 Assembly and Composition Layers

11.1 LayerStack

The stack assembly API represents the full simulator as an ordered composition

$$y = (f_L \circ f_{L-1} \circ \cdots \circ f_1 \circ s)(\emptyset), \quad (68)$$

where s is a source layer with no wave input and each f_ℓ is a unary map on either a wave or a derived measurement tensor. The implementation also allows the intermediate states

$$z_0 = s(\emptyset), \quad (69)$$

$$z_\ell = f_\ell(z_{\ell-1}), \quad \ell = 1, \dots, L, \quad (70)$$

to be returned explicitly for debugging, supervision, or auxiliary losses.

11.2 PotentialSliceInteraction

The potential-slice interaction layer binds a potential-producing module to a multislice propagator:

$$\psi_{\text{out}} = \mathcal{M}\left(\psi_{\text{in}}, \{V_j\}_{j=1}^{N_s}\right), \quad (71)$$

where $\mathcal{M}$ denotes the multislice recursion and the potential module produces either a slice stack $\{V_j\}_{j=1}^{N_s}$ directly or a single projected potential $V(\mathbf{r})$ that is distributed uniformly as

$$V_j(\mathbf{r}) = \frac{1}{N_s} V(\mathbf{r}). \quad (72)$$

11.3 ScanLayer

The scan layer converts a reference wave into a batch of shifted waves over a position set $\mathcal{S} = \{\Delta \mathbf{r}_m\}_{m=1}^M$:

$$\psi^{(m)}(\mathbf{r}) = \mathcal{T}_{\Delta \mathbf{r}_m} \psi(\mathbf{r}), \quad m = 1, \dots, M, \quad (73)$$

where $\mathcal{T}_{\Delta \mathbf{r}_m}$ is the implemented Fourier-shift operator from the scan section above. The output is therefore a stacked tensor

$$\Psi_{\text{scan}} = \{\psi^{(m)}\}_{m=1}^M. \quad (74)$$

11.4 DetectorReadout and BatchDetectorReadout

For a detector functional $\mathcal{D}$, the unary detector readout is simply

$$y = \mathcal{D}[\psi]. \quad (75)$$

For a leading scan or ensemble batch, the batch detector path applies the same detector independently:

$$y_m = \mathcal{D}[\psi^{(m)}], \quad m = 1, \dots, M. \quad (76)$$

This keeps the whole assembled simulator differentiable as a map from source and specimen parameters to either scalar signals, image stacks, or diffraction data.

11.5 Structured experiment assembly

The experiment-builder layer defines a parameter object

$$\Theta = (\Theta_{\text{src}}, \Theta_{\text{ms}}, \Theta_{\text{pot}}, \Theta_{\text{scan}}, \Theta_{\text{det}}, \Theta_{\text{coh}}), \quad (77)$$

where omitted blocks are allowed for modes that do not require them. The builder then instantiates a composed simulator

$$\mathcal{E}_\Theta = \mathcal{R}_{\Theta_{\text{coh}}, \Theta_{\text{det}}} \circ \mathcal{S}_{\Theta_{\text{scan}}} \circ \mathcal{M}_{\Theta_{\text{ms}}, \Theta_{\text{pot}}} \circ \mathcal{Q}_{\Theta_{\text{src}}}, \quad (78)$$

with the conventions that $\mathcal{S}$ becomes the identity if no scan is present, and $\mathcal{R}$ reduces to the identity, detector-only readout, or coherence-aware readout depending on the selected experiment mode.

For a plane-wave source configuration,

$$\mathcal{Q}_{\Theta_{\text{src}}}(\emptyset) = A\mathbf{1}, \quad (79)$$

whereas for a probe-forming source configuration,

$$\mathcal{Q}_{\Theta_{\text{src}}}(\emptyset) = \mathcal{F}^{-1} \{H_{\Theta_{\text{src}}}(\mathbf{k})\}. \quad (80)$$

The resulting experiment map therefore provides a differentiable parameter-to-output operator

$$y = \mathcal{E}_\Theta(\emptyset), \quad (81)$$

which is the object intended for downstream gradient-based calibration, inverse problems, and external training pipelines.

11.6 Multi-detector experiment branches

For realistic STEM/TEM experiments the detector block may itself be a family of simultaneous readouts

$$\mathcal{D} = \{\mathcal{D}_q\}_{q \in \mathcal{Q}}, \quad (82)$$

where $\mathcal{Q}$ is a set of user-defined detector names such as annular, segmented, or pixelated channels. The experiment readout then returns a named output map

$$y_q = \mathcal{D}_q[\psi], \quad q \in \mathcal{Q}, \quad (83)$$

or, for scanned experiments,

$$y_q^{(m)} = \mathcal{D}_q[\psi^{(m)}], \quad m = 1, \dots, M. \quad (84)$$

With coherence averaging enabled in the ensemble-averaging mode, each named branch becomes

$$y_q^{(m)} = \sum_{\ell=1}^L w_\ell \mathcal{D}_q[\psi_\ell^{(m)}], \quad (85)$$

with normalized coherence weights w_ℓ . This is the semantics used by the Torch-native multi-detector experiment builder and plotting surface.

For diagonal mixed coherence, the experiment source is replaced by coherent modes $\psi_\ell^{(0)}$ generated by the diagonal mixed-coherence factor decomposition, the elastic stack propagates each mode independently,

$$\psi_\ell^{(m)} = \mathcal{M}_{\Theta_{\text{ms}}, \Theta_{\text{pot}}} \mathcal{S}_{\mathbf{r}^{(m)}} \psi_\ell^{(0)}, \quad (86)$$

and the detector outputs are reduced incoherently after propagation:

$$y_q^{(m)} = \sum_{\ell=1}^L \mathcal{D}_q[\psi_\ell^{(m)}]. \quad (87)$$

If no detector is configured, the experiment returns the incoherent real-space intensity

$$I^{(m)}(\mathbf{r}) = \sum_{\ell=1}^L \left| \psi_\ell^{(m)}(\mathbf{r}) \right|^2. \quad (88)$$

11.7 Mode presets

The preset experiment surfaces are typed restrictions of the general experiment map $\mathcal{E}_\Theta$.

HRTEM preset. The HRTEM preset omits scanning and uses

$$y = \mathcal{D}[\mathcal{M}_{\Theta_{\text{ms}}, \Theta_{\text{pot}}} \mathcal{Q}_{\Theta_{\text{src}}}(\emptyset)], \quad (89)$$

where $\mathcal{D}$ is typically a pixelated image-plane or diffraction-plane readout.

STEM preset. The STEM preset requires a probe-forming source and scan path:

$$y^{(m)} = \mathcal{D}[\mathcal{M}_{\Theta_{\text{ms}}, \Theta_{\text{pot}}} \mathcal{T}_{\Delta \mathbf{r}_m} \mathcal{Q}_{\Theta_{\text{src}}}(\emptyset)]. \quad (90)$$

Diffraction preset. The diffraction preset uses a pixelated detector branch by default:

$$y^{(m)}(\mathbf{k}) = \left| \mathcal{F} \{ \mathcal{M}_{\Theta_{\text{ms}}, \Theta_{\text{pot}}} \mathcal{T}_{\Delta \mathbf{r}_m} \mathcal{Q}_{\Theta_{\text{src}}}(\emptyset) \} \right|^2, \quad (91)$$

with the scan operator removed when no scan path is configured.

11.8 Simulation-result wrapper

The simulator-facing run helper does not change the computed output; it packages the same final tensor or named detector map together with metadata and optional intermediate states. If the layer stack has states

$$\{z_\ell\}_{\ell=0}^L, \quad (92)$$

then the wrapped result stores

$$y = z_L, \quad (93)$$

$$\mathcal{I} = \{z_\ell\}_{\ell=0}^L, \quad (94)$$

when intermediate capture is requested, and otherwise stores only y together with metadata such as experiment mode, scan/coherence flags, and detector names. Plotting is defined as a visualization map

$$\mathcal{P} : y \mapsto \text{matplotlib panels}, \quad (95)$$

where each detector branch or scan-indexed slice is rendered as a labeled panel.

11.9 Post-detector measurement pipelines

The experiment surface may append a post-detector pipeline

$$\mathcal{R} = \mathcal{R}_P \circ \cdots \circ \mathcal{R}_2 \circ \mathcal{R}_1 \quad (96)$$

after detector readout. The resulting experiment output becomes

$$\tilde{y} = \mathcal{R}(y), \quad (97)$$

or, for named detector branches,

$$\tilde{y}_q = \mathcal{R}_q(y_q), \quad q \in \mathcal{Q}, \quad (98)$$

where each branch may have its own configured sequence.

Implemented examples include:

$$\mathcal{R}_{\text{blur}}(I) = G_\sigma * I, \quad (99)$$

$$\mathcal{R}_{\text{MTF}}(I) = \mathcal{F}^{-1} \left\{ \sqrt{K_{\text{MTF}}(\mathbf{k})} \mathcal{F}\{I\} \right\}, \quad (100)$$

$$\mathcal{R}_{\text{Poisson}}(I) \sim \text{Poisson}(dI), \quad (101)$$

$$\mathcal{R}_{\text{COM}}(I) = \left(\frac{\sum_{\mathbf{k}} k_y I(\mathbf{k})}{\sum_{\mathbf{k}} I(\mathbf{k})}, \frac{\sum_{\mathbf{k}} k_x I(\mathbf{k})}{\sum_{\mathbf{k}} I(\mathbf{k})} \right). \quad (102)$$

This is the semantics used by the Torch-native experiment postprocessing layer.

11.10 Propagation-backend selection

The experiment multislice configuration may choose between distinct propagation operators while preserving the same higher-level assembly semantics. Denote the chosen propagation backend by $\mathcal{M}^{(b)}$, where

$$b \in \{\text{fourier}, \text{realspace}\}. \quad (103)$$

The experiment map then uses

$$y = \mathcal{D} \left[\mathcal{M}_{\Theta_{\text{ms}}, \Theta_{\text{pot}}}^{(b)} \mathcal{Q}_{\Theta_{\text{src}}}(\emptyset) \right]. \quad (104)$$

For $b = \text{fourier}$ this is the reciprocal-space Fresnel multislice recursion described above. For $b = \text{realspace}$ it is the truncated exponential-series real-space propagator already implemented in the low-level stack. The experiment layer therefore selects among low-level propagators without changing the surrounding source, scan, detector, or measurement-pipeline structure.

For $b = \text{prism}$, the scanned probe experiment uses an S-matrix reduction path:

$$S_p(\mathbf{r}) = \mathcal{M}_{\Theta_{\text{ms}}, \Theta_{\text{pot}}}^{(\text{fourier})} \phi_p(\mathbf{r}), \quad (105)$$

$$\psi^{(m)}(\mathbf{r}) = \sum_{p=1}^P c_p \exp[-2\pi i \mathbf{k}_p \cdot \Delta \mathbf{r}_m] S_p(\mathbf{r}), \quad (106)$$

where ϕ_p are parent plane waves, $\mathbf{k}_p$ are the selected parent wave vectors, and c_p are the probe coefficients sampled from the CTF grid. For a scan configuration, the phase factor produces one reduced wave per scan position $\Delta \mathbf{r}_m$; without a scan configuration, the current experiment layer reduces the S-matrix once at the origin. This is the current experiment-level PRISM path exposed by the Torch-native simulator surface.

For $b = \text{bloch}$, the experiment currently exposes a diffraction-style path driven by structure factors:

$$\mathbf{c}(t) = S_{\text{Bloch}}(t) \mathbf{c}_0, \quad (107)$$

$$I_j = |c_j(t)|^2, \quad (108)$$

where $\mathbf{c}_0$ is the incident plane-wave coefficient vector and S_{Bloch} is the Bloch scattering matrix from the low-level model. The experiment layer rasterizes $\{I_j\}$ onto the configured reciprocal-space image grid at the corresponding projected reciprocal-lattice coordinates. Reciprocal-intensity detector readout may then be applied directly to this rasterized diffraction image for the current pixelated-reciprocal, annular, flexible-annular, and segmented detector families. Alternatively, the complex coefficients $c_j(t)$ themselves may be rasterized onto the reciprocal grid for reciprocal-wave output. This is intentionally a constrained diffraction-oriented surface rather than full Bloch feature parity.

11.11 Inelastic branch

The experiment layer may append a constrained inelastic branch after elastic propagation, and this branch may also be followed by the current coherence-averaging readout path:

$$\psi_{\text{el}} = \mathcal{M}_{\Theta_{\text{ms}}, \Theta_{\text{pot}}} \mathcal{Q}_{\Theta_{\text{src}}}(\emptyset), \quad (109)$$

followed by an inelastic operator $\mathcal{T}_{\text{inel}}$.

For channel-producing transition potentials,

$$\psi_c^{\text{inel}}(\mathbf{r}) = T_c(\mathbf{r}) \psi_{\text{el}}(\mathbf{r}), \quad (110)$$

and the experiment output is either the stack $\{\psi_c^{\text{inel}}\}_c$ or, when a detector branch is configured, the recursively reduced detector outputs

$$y_c = \mathcal{D}[\psi_c^{\text{inel}}]. \quad (111)$$

For plasmon weighted-intensity mode, the branch evaluates shifted inelastic members

$$\psi_\ell^{\text{pl}}(\mathbf{r}) = \mathcal{S}_{\Delta\mathbf{r}_\ell} \psi_{\text{el}}(\mathbf{r}), \quad (112)$$

and returns

$$I_{\text{pl}}(\mathbf{r}) = \sum_\ell w_\ell \left| \psi_\ell^{\text{pl}}(\mathbf{r}) \right|^2. \quad (113)$$

If a coherence model is configured, detector or intensity outputs are reduced over coherent members after the inelastic branch using the same ensemble-weighted or diagonal-MCF incoherent reduction as the elastic experiment path. This is the current constrained inelastic experiment surface exposed by TorchTEM.

11.12 Magnetic branch

The experiment layer may also append a projected magnetic interaction after elastic propagation:

$$\psi_{\text{mag}} = \mathcal{P}_A(A_{xy}, \psi_{\text{el}}), \quad (114)$$

where A_{xy} is a projected vector potential from the magnetic builder and $\mathcal{P}_A$ is the projected Pauli interaction currently implemented as

$$\psi_{\text{mag}} = \psi_{\text{el}} + \eta A_{xy} \cdot \nabla_{xy} \psi_{\text{el}}, \quad (115)$$

with configurable complex prefactor η . The resulting magnetic wave may then feed the usual detector and postprocessing branches. This is the current constrained magnetic experiment surface exposed by TorchTEM.

11.13 Image-forming detector branch

The Torch-native detector family now also includes a real-space image-forming branch. Given an exit wave $\psi_{\text{exit}}(\mathbf{r})$, the detector may optionally apply an objective-lens transfer function

$$H(\mathbf{k}) = A(\mathbf{k}) \exp[-i\chi(\mathbf{k})], \quad (116)$$

where A is the aperture and χ is the aberration phase polynomial from the CTF layer already defined above. The image-plane wave is then

$$\psi_{\text{img}}(\mathbf{r}) = \mathcal{F}^{-1} \{ H(\mathbf{k}) \mathcal{F} \{ \psi_{\text{exit}}(\mathbf{r}) \} \}. \quad (117)$$

The detector may return several real-space channels:

$$I(\mathbf{r}) = |\psi_{\text{img}}(\mathbf{r})|^2, \quad (118)$$

$$\varphi(\mathbf{r}) = \arg \psi_{\text{img}}(\mathbf{r}), \quad (119)$$

$$R(\mathbf{r}) = \Re \psi_{\text{img}}(\mathbf{r}), \quad (120)$$

$$J(\mathbf{r}) = \Im \psi_{\text{img}}(\mathbf{r}), \quad (121)$$

or the complex image-plane wave itself. In experiment-builder notation, this makes it possible to express HRTEM-style image formation as a detector branch

$$y = \mathcal{D}_{\text{img}} [\mathcal{M}_{\Theta_{\text{ms}}, \Theta_{\text{pot}}} \mathcal{Q}_{\Theta_{\text{src}}}(\emptyset)] \quad (122)$$

instead of applying an imaging CTF manually outside the assembled simulator graph.

11.14 Direct waves detector branch

The detector family also includes a direct wave readout branch for optimization or reconstruction pipelines that require complex fields instead of already reduced image intensities. Given an exit wave $\psi_{\text{exit}}(\mathbf{r})$, the detector may either return the real-space wave directly,

$$y_{\text{exit}}(\mathbf{r}) = \psi_{\text{exit}}(\mathbf{r}), \quad (123)$$

or the reciprocal-space wave

$$y_{\text{rec}}(\mathbf{k}) = \mathcal{F}\{\psi_{\text{exit}}(\mathbf{r})\}. \quad (124)$$

If a target detector grid $\mathbf{N}' = (N'_y, N'_x)$ is requested, the returned wave is resampled by FFT cropping/interpolation:

$$\tilde{y} = \mathcal{I}_{\mathbf{N}'}[y], \quad (125)$$

where $\mathcal{I}_{\mathbf{N}'}$ denotes the Torch-native Fourier interpolation operator already implemented in the support layer. This yields a detector branch

$$y = \mathcal{D}_{\text{waves}} [\mathcal{M}_{\Theta_{\text{ms}}, \Theta_{\text{pot}}} \mathcal{Q}_{\Theta_{\text{src}}}(\emptyset)] \quad (126)$$

that preserves the complex field for downstream differentiable pipelines.

11.15 Flexible annular detector branch

The detector family also includes a radial-binning annular detector that preserves annular integration flexibility after simulation. Let the diffraction intensity be

$$I(\mathbf{k}) = |\mathcal{F}\{\psi(\mathbf{r})\}|^2, \quad (127)$$

and define the shifted scattering-angle magnitude

$$\alpha_{\Delta}(\mathbf{k}) = \sqrt{(\alpha_y(\mathbf{k}) - \Delta\alpha_y)^2 + (\alpha_x(\mathbf{k}) - \Delta\alpha_x)^2}, \quad (128)$$

in mrad. For inner angle α_{in} , optional outer angle α_{out} , and radial step size $\Delta\alpha$, the detector defines radial bin indices

$$b(\mathbf{k}) = \left\lfloor \frac{\alpha_{\Delta}(\mathbf{k}) - \alpha_{\text{in}}}{\Delta\alpha} \right\rfloor, \quad (129)$$

restricted to pixels satisfying

$$\alpha_{\text{in}} \leq \alpha_{\Delta}(\mathbf{k}) < \alpha_{\text{out}}. \quad (130)$$

The detector output is then the vector of radial annular sums

$$y_n = \sum_{\mathbf{k}: b(\mathbf{k})=n} I(\mathbf{k}), \quad (131)$$

for $n = 0, \dots, N_r - 1$, where

$$N_r = \left\lfloor \frac{\alpha_{\text{out}} - \alpha_{\text{in}}}{\Delta\alpha} \right\rfloor. \quad (132)$$

This provides a differentiable analogue of the original flexible annular detector: the simulation produces a radial-binned detector signal once, and downstream annular integrations may be chosen afterward without recomputing the forward wave propagation. The Torch-native implementation now also supports off-axis detector-center shifts $\Delta\alpha$ in this branch. The corresponding detector-region helper returns the integer-valued radial bin map $b(\mathbf{k})$ with invalid pixels marked outside the accepted annular support. An additional helper returns the family of per-bin masks $\{\mathbf{1}_{b(\mathbf{k})=n}\}_n$, represented as separate 2D region planes with NaN outside the active bin so radial detector regions can be visualized directly. Because the detector output is the radial-bin vector $\{y_n\}$, a post-hoc annular integration over limits (α_1, α_2) is implemented as

$$D_{\text{flex}}(\alpha_1, \alpha_2) = \sum_{n=n_1}^{n_2-1} y_n, \quad (133)$$

where

$$n_1 = \left\lfloor \frac{\alpha_1 - \alpha_{\text{in}}}{\Delta\alpha} \right\rfloor, \quad (134)$$

$$n_2 = \left\lfloor \frac{\alpha_2 - \alpha_{\text{in}}}{\Delta\alpha} \right\rfloor. \quad (135)$$

For limits aligned with the radial bin boundaries, this matches the corresponding fixed annular detector integral exactly up to floating-point arithmetic.

11.16 Fixed-stack HRTEM control-vector branch

TorchTEM also includes a fixed-topology HRTEM branch for inverse microscope tuning with a constant specimen. Let

$$\mathbf{u} \in \mathbb{R}^{12} \quad (136)$$

denote the control vector

$$\mathbf{u} = (I_{c1}, I_{c2}, s_{c,x}, s_{c,y}, f_o, s_{o,x}, s_{o,y}, \gamma_{30}, \gamma_{32,x}, \gamma_{32,y}, \gamma_{34,x}, \gamma_{34,y}), \quad (137)$$

where the entries represent two condenser excitations, condenser stigmator components, objective focus, objective stigmator components, and simple corrector controls. For batched inference or optimization, the implementation also accepts a tensor

$$U \in \mathbb{R}^{B_1 \times \dots \times B_m \times 12}, \quad (138)$$

which is interpreted as a batch of control vectors with arbitrary leading batch shape. The current fixed-stack surface also groups these entries into stage blocks

$$\mathbf{u} = (\mathbf{u}_{\text{cond}}, \mathbf{u}_{\text{obj}}, \mathbf{u}_{\text{corr}}), \quad (139)$$

with

$$\mathbf{u}_{\text{cond}} \in \mathbb{R}^4, \quad (140)$$

$$\mathbf{u}_{\text{obj}} \in \mathbb{R}^3, \quad (141)$$

$$\mathbf{u}_{\text{corr}} \in \mathbb{R}^5, \quad (142)$$

corresponding respectively to the condenser, objective, and corrector stage vectors. In the implementation these stage blocks may be extracted or replaced independently before running the full differentiable HRTEM forward model. For inverse fitting, the current stack also supports an active subspace

$$\mathbf{u}_{\mathcal{A}} \in \mathbb{R}^{|\mathcal{A}|}, \quad (143)$$

which replaces only a selected subset of control entries while the remaining entries stay fixed at a baseline microscope state. When bounds $\ell_i < u_i < h_i$ are supplied for an active coordinate, the trainable latent parameter $z_i \in \mathbb{R}$ is mapped to the physical control value by

$$u_i = \ell_i + (h_i - \ell_i) \sigma(z_i), \quad (144)$$

with σ the logistic sigmoid. Unbounded active coordinates are represented directly. This keeps the differentiable fitting path inside a chosen physically plausible parameter window without changing the fixed-stack forward model itself.

The current demo implementation decodes this vector into source-side and image-side aberration coefficients through an explicit affine calibration map. For the source branch, the most important terms are

$$C_{10}^{(\text{src})} = -65I_{c1} - 28I_{c2}, \quad (145)$$

$$a_{12,x}^{(\text{src})} = 24s_{c,x} + 7s_{c,y}, \quad (146)$$

$$a_{12,y}^{(\text{src})} = 7s_{c,x} - 24s_{c,y}, \quad (147)$$

$$C_{12}^{(\text{src})} = \sqrt{\left(a_{12,x}^{(\text{src})}\right)^2 + \left(a_{12,y}^{(\text{src})}\right)^2}, \quad (148)$$

$$\phi_{12}^{(\text{src})} = \frac{1}{2} \text{atan2}\left(a_{12,y}^{(\text{src})}, a_{12,x}^{(\text{src})}\right), \quad (149)$$

$$C_{21}^{(\text{src})} = 14(I_{c1} - I_{c2}). \quad (150)$$

For the image-forming branch, the current demo map uses

$$C_{10}^{(\text{img})} = 110f_o + 6(I_{c1} - I_{c2}), \quad (151)$$

$$a_{12,x}^{(\text{img})} = 18s_{o,x} - 900\gamma_{32,x}, \quad (152)$$

$$a_{12,y}^{(\text{img})} = 18s_{o,y} - 900\gamma_{32,y}, \quad (153)$$

$$C_{12}^{(\text{img})} = \sqrt{\left(a_{12,x}^{(\text{img})}\right)^2 + \left(a_{12,y}^{(\text{img})}\right)^2}, \quad (154)$$

$$\phi_{12}^{(\text{img})} = \frac{1}{2} \text{atan2}\left(a_{12,y}^{(\text{img})}, a_{12,x}^{(\text{img})}\right), \quad (155)$$

$$C_{30}^{(\text{img})} = 1.2 \times 10^4 - 8.5 \times 10^3 \gamma_{30}, \quad (156)$$

$$a_{32,x}^{(\text{img})} = -2200\gamma_{32,x}, \quad (157)$$

$$a_{32,y}^{(\text{img})} = -2200\gamma_{32,y}, \quad (158)$$

$$C_{32}^{(\text{img})} = \sqrt{\left(a_{32,x}^{(\text{img})}\right)^2 + \left(a_{32,y}^{(\text{img})}\right)^2}, \quad (159)$$

$$\phi_{32}^{(\text{img})} = \frac{1}{2} \text{atan2}\left(a_{32,y}^{(\text{img})}, a_{32,x}^{(\text{img})}\right), \quad (160)$$

$$a_{34,x}^{(\text{img})} = -1400\gamma_{34,x}, \quad (161)$$

$$a_{34,y}^{(\text{img})} = -1400\gamma_{34,y}, \quad (162)$$

$$C_{34}^{(\text{img})} = \sqrt{\left(a_{34,x}^{(\text{img})}\right)^2 + \left(a_{34,y}^{(\text{img})}\right)^2}, \quad (163)$$

$$\phi_{34}^{(\text{img})} = \frac{1}{4} \text{atan2}\left(a_{34,y}^{(\text{img})}, a_{34,x}^{(\text{img})}\right). \quad (164)$$

All higher coefficients not listed above are currently set to zero in this fixed-stack demo surface.

The source wave is then generated from the decoded source coefficients by evaluating the source transfer function

$$H_{\text{src}}(\mathbf{k}; \mathbf{u}), \quad (165)$$

normalizing it in reciprocal space,

$$\widehat{\psi}_{\text{src}}(\mathbf{k}; \mathbf{u}) = \frac{H_{\text{src}}(\mathbf{k}; \mathbf{u})}{\|H_{\text{src}}(\cdot; \mathbf{u})\|_2}, \quad (166)$$

and applying the inverse Fourier transform,

$$\psi_{\text{src}}(\mathbf{r}; \mathbf{u}) = \mathcal{F}^{-1}\left\{\widehat{\psi}_{\text{src}}(\mathbf{k}; \mathbf{u})\right\}. \quad (167)$$

The current implementation then shifts this illumination to the center of the real-space grid before propagation through the specimen. For the validated fixed-stack example, the source semiangle cutoff is now chosen large enough that the source aperture spans multiple reciprocal pixels on the simulation grid. In the current example this means a ‘5.0 mrad’ source aperture rather than the earlier effectively single-pixel regime, so condenser-side controls now produce a nontrivial pre-specimen wave and a genuinely source-sensitive exit-wave objective.

For a fixed specimen potential stack $\{V_n\}_{n=1}^{N_s}$, the exit wave is

$$\psi_{\text{exit}}(\mathbf{r}; \mathbf{u}) = \mathcal{M}_{\Theta_{\text{ms}}, \Theta_{\text{pot}}}[\psi_{\text{src}}(\mathbf{r}; \mathbf{u})]. \quad (168)$$

The image-plane wave and image intensity are then formed with the decoded image-side CTF,

$$\psi_{\text{img}}(\mathbf{r}; \mathbf{u}) = \mathcal{F}^{-1}\{H_{\text{img}}(\mathbf{k}; \mathbf{u}) \mathcal{F}\{\psi_{\text{exit}}(\mathbf{r}; \mathbf{u})\}\}, \quad (169)$$

$$I_{\text{img}}(\mathbf{r}; \mathbf{u}) = |\psi_{\text{img}}(\mathbf{r}; \mathbf{u})|^2, \quad (170)$$

and the momentum-plane detector output is

$$I_{\text{diff}}(\mathbf{k}; \mathbf{u}) = |\mathcal{F}\{\psi_{\text{exit}}(\mathbf{r}; \mathbf{u})\}|^2. \quad (171)$$

The branch therefore emits the tuple

$$y(\mathbf{u}) = (\psi_{\text{src}}(\mathbf{u}), I_{\text{img}}(\mathbf{u}), I_{\text{diff}}(\mathbf{u}), \psi_{\text{exit}}(\mathbf{u})), \quad (172)$$

so the pre-specimen source wave, reduced detector data, and the full complex exit wave remain available to downstream optimization pipelines. For a batched input tensor U , the implementation applies the same forward model to each control slice independently and preserves the leading batch shape:

$$y(U) \in \mathbb{C}^{B_1 \times \dots \times B_m \times N_y \times N_x} \quad (173)$$

for the complex exit-wave branch, with the corresponding real-valued detector outputs carrying the same leading batch dimensions.

For the current fitting example, the primary optimization target is image contrast rather than raw intensity:

$$I_{\text{contrast}}(\mathbf{r}; \mathbf{u}) = \frac{I_{\text{img}}(\mathbf{r}; \mathbf{u})}{I_{\text{img}}^*(\cdot)} - 1, \quad (174)$$

and the same branch also supports a normalized diffraction metric

$$D_{\text{norm}}(\mathbf{k}; \mathbf{u}) = \frac{D(\mathbf{k}; \mathbf{u})}{\sum_{\mathbf{k}'} D(\mathbf{k}'; \mathbf{u})}. \quad (175)$$

With diffraction weight $\lambda_D \geq 0$, the demo loss becomes

$$\mathcal{L}(\mathbf{u}) = \frac{1}{N_I} \sum_{\mathbf{r}} (I_{\text{contrast}}(\mathbf{r}; \mathbf{u}) - I_{\text{contrast}}^*(\mathbf{r}))^2 + \lambda_D \frac{1}{N_D} \sum_{\mathbf{k}} (D_{\text{norm}}(\mathbf{k}; \mathbf{u}) - D_{\text{norm}}^*(\mathbf{k}))^2. \quad (176)$$

This should be interpreted as a differentiable inverse-tuning demonstration, not as a claim of unique parameter recovery for the current overparameterized control map. For controls that mainly act in the image-forming branch after the specimen exit wave, the diffraction contribution can remain nearly constant. Reciprocal-space fitting is more informative when the optimized control subset includes source-side parameters that change the exit-wave illumination itself. The current validated HRTEM fitting example therefore screens a mixed source/image plane (I_{c1}, f_o) rather than a purely post-specimen image-side control plane, while the currently informative live-validated exit-wave scoring plane is the source-side pair (I_{c1}, I_{c2}) . Using the target-image mean as the shared contrast reference also makes the score surface less sensitive to tiny mean-level numerical differences between otherwise matching low-contrast images.

For coarse candidate screening before gradient refinement, the same branch now also supports batched candidate scoring. Given a candidate tensor

$$U \in \mathbb{R}^{B \times 12} \quad (177)$$

and a target image $I^\star(\mathbf{r})$, the simulator evaluates all candidates and forms the batched image metric

$$M_b(\mathbf{r}) = \begin{cases} I_{\text{contrast}}(\mathbf{r}; U_b), & \text{if contrast scoring is enabled,} \\ I_{\text{img}}(\mathbf{r}; U_b), & \text{otherwise,} \end{cases} \quad (178)$$

with corresponding target metric

$$M^\star(\mathbf{r}) = \begin{cases} I_{\text{contrast}}^\star(\mathbf{r}), & \text{if contrast scoring is enabled,} \\ I^\star(\mathbf{r}), & \text{otherwise.} \end{cases} \quad (179)$$

with the target contrast defined using the same reference mean $\overline{I_{\text{img}}^\star(\cdot)}$ for both the target and all candidate predictions. If the inverse target also includes the full complex exit wave $\psi_{\text{exit}}^\star(\mathbf{r})$, the simulator can first apply a global phase alignment

$$\hat{\psi}_{b,\text{exit}}(\mathbf{r}) = \exp\left(-i \arg \sum_{\mathbf{r}'} \overline{\psi_{\text{exit}}^\star(\mathbf{r}')} \psi_{b,\text{exit}}(\mathbf{r}')\right) \psi_{b,\text{exit}}(\mathbf{r}), \quad (180)$$

and then form the exit-wave residual

$$L_{\psi,b} = \frac{1}{N_\psi} \sum_{\mathbf{r}} \left| \hat{\psi}_{b,\text{exit}}(\mathbf{r}) - \psi_{\text{exit}}^\star(\mathbf{r}) \right|^2. \quad (181)$$

If the inverse target instead or additionally includes the pre-specimen source wave $\psi_{\text{src}}^\star(\mathbf{r})$, the simulator can apply the same global phase-alignment convention

$$\hat{\psi}_{b,\text{src}}(\mathbf{r}) = \exp\left(-i \arg \sum_{\mathbf{r}'} \overline{\psi_{\text{src}}^\star(\mathbf{r}')} \psi_{b,\text{src}}(\mathbf{r}')\right) \psi_{b,\text{src}}(\mathbf{r}), \quad (182)$$

and then form the source-wave residual

$$L_{\text{src},b} = \frac{1}{N_\psi} \sum_{\mathbf{r}} \left| \hat{\psi}_{b,\text{src}}(\mathbf{r}) - \psi_{\text{src}}^\star(\mathbf{r}) \right|^2. \quad (183)$$

The candidate score is then the per-candidate mean squared residual, optionally with the same diffraction term, the exit-wave term, and the source-wave term,

$$s_b = \frac{1}{N_I} \sum_{\mathbf{r}} (M_b(\mathbf{r}) - M^\star(\mathbf{r}))^2 + \lambda_D \frac{1}{N_D} \sum_{\mathbf{k}} \left(\tilde{D}_b(\mathbf{k}) - \tilde{D}^\star(\mathbf{k}) \right)^2 + \lambda_\psi L_{\psi,b} + \lambda_{\text{src}} L_{\text{src},b}. \quad (184)$$

Here $\tilde{D}$ denotes either the raw diffraction intensity or the normalized diffraction metric depending on the selected scoring mode. This provides a differentiable screening stage that can rank many microscope-state proposals against one fixed target image before or alongside subsequent local optimization. For local refinement after coarse screening, the simulator also exposes an iterative optimizer surface that repeatedly evaluates the fixed-stack forward model, computes the same image, diffraction, and optional wavefield losses, and updates the active control parameters with a Torch optimizer. The returned result object records the loss history together with the initial outputs, final outputs, final control vector, and the per-step exit-wave and source-wave loss histories when those terms are enabled, so the inverse-tuning path is available as a reusable differentiable API rather than only as example-script logic. Above that, the simulator also exposes a coarse-to-fine orchestration surface for Cartesian named-parameter grids: it screens the candidate grid,

selects the best screened control vector, instantiates a fresh fixed-stack simulator at that screened state, and then runs the same differentiable local refinement on top of it. This promotes the whole grid-screening plus bounded-refinement workflow to a reusable layer instead of leaving the orchestration inside one example script. Because the implementation now also accepts named stage blocks directly, the same candidate screening may be viewed as acting on a stage-parameter family

$$U_{\text{stage}} = (U_{\text{cond}}, U_{\text{obj}}, U_{\text{corr}}), \quad (185)$$

which is first assembled into the full candidate tensor U and then scored by the same formula for s_b . In other words, stage-wise candidate screening is not a different physical model; it is a different parameterization of the same fixed-stack HRTEM forward map. The simulator additionally exposes the corresponding best-candidate selector

$$b^* = \arg \min_b s_b, \quad (186)$$

and returns both the index b^* and the associated control vector

$$\mathbf{u}^* = U_{b^*}, \quad (187)$$

together with the image, diffraction, and exit-wave outputs for that best-scoring candidate. This yields a direct coarse-to-fine workflow: screen a batched family of microscope-state proposals, extract $\mathbf{u}^*$, and continue refinement from that candidate by gradient-based optimization if needed. For workflows that keep several promising candidates, the same score tensor may be ranked by

$$b_1^*, \dots, b_k^* = \arg \text{topk}_b \min s_b, \quad (188)$$

with associated vectors

$$\mathbf{u}_{(1)}^*, \dots, \mathbf{u}_{(k)}^*, \quad (189)$$

returned in ascending score order. This allows downstream optimization to branch from the best few microscope-state proposals instead of only the single global minimum over the screened batch. When the candidates come from a Cartesian parameter scan such as

$$I_{c1} \in \{I_{c1}^{(1)}, \dots, I_{c1}^{(M)}\}, \quad f_o \in \{f_o^{(1)}, \dots, f_o^{(N)}\}, \quad (190)$$

the implementation first builds the corresponding parameter grid tensor and then applies the same score map s_b , best-candidate selector b^* , and top- k ranking operators. In other words, grid screening is the same coarse-search primitive as the arbitrary batched candidate case, but with a structured Cartesian parameterization of the proposal family.